

JavaScript Alternatives

<Open web standards are about freedom and access, there is one freedom that open web developers are not afforded by the standards, and that is the choice of languages used for web development> **By Kshitij Sobti**

If you're a web developer, HTML, CSS and JavaScript are your bread and butter. While open standards are awesome, the very fact that they're standard, or, in other words, unchanging means that there will be those who love them and those who hate them.

Native developers are used to having a plethora of choices when it comes to development: Assembly, C, C++, C#, VisualBasic, Objective-C, Ada, Java, Python, Ruby, Lisp, Perl or, heck even JavaScript! Native developers are spoiled by choice, and all web developers have is JavaScript. Or do they?

While browsers don't support any scripting language other than JavaScript, web developers have found ways around this limitation to give them access to a growing number of languages. Recently, Google announced a JavaScript alternative called Dart that they'd been developing. Let's look at some alternatives to JavaScript.

Before we get there, ever wondered how such languages work if the browser doesn't understand them? Well, all these languages "compile" to JavaScript. So while you can code your application in any one of these languages using a preprocessing tool, whatever code you write will be converted to equivalent JavaScript. It is this "compiled" JavaScript that you will finally include in your web page so it can run in the browser.

It is important to understand that these languages don't add any new "features" to JavaScript. What you can do is still limited by the limitation of the browser and JavaScript, but how you code it will differ.

CoffeeScript

This is one of the more popular and active JavaScript alternatives. It is a favoured scripting language of the Ruby on Rails web development framework.

The CoffeeScript compiler is written in CoffeeScript itself, and is available as a package for Node.js. In fact, you don't even need to compile CoffeeScript code to JavaScript before running it in the browser. You can just include the compiler along with the CoffeeScript code to have the compilation performed in the browser itself. This is great for testing, but is something that obviously doesn't make sense in a production website.

CoffeeScript has been inspired by Ruby, Python and Haskell, so you will find borrowed features and syntax from those languages. The code sample is fairly reminiscent of the above languages for those familiar with them:

```
square = (n) -> i * i for i in [1..n]
document.write square 10
```

This code defines a function called square that returns an array containing the squares of the first 'n' numbers. After that we use that function to write the squares of the first 10 numbers to the page.

CoffeeScript code is much more compact and to the point, and so is the JavaScript code this generates — try for yourself, CoffeeScript is included in the DVD. CoffeeScript has numerous syntax improvements for working with classes, objects, arrays, loops and so much more. It is well supported, and the JavaScript it produces has equivalent, or sometimes even better, performance as hand-written JavaScript.

JavaScript is included in the DVD. CoffeeScript has numerous syntax improvements for working with classes, objects, arrays, loops and so much more. It is well supported, and the JavaScript it produces has equivalent, or sometimes even better, performance as hand-written JavaScript.

ClojureScript

Clojure (with a 'j') is a language that compiles to run on the JVM, so it can be considered an alternative to Java. ClojureScript is a subset of the Clojure language that can be compiled to JavaScript and run in the browser.

Do not confuse Clo(j)ure with Clo(s)ure, which are a set of tools by Google for optimising JavaScript. In fact, Google Closure tools are a big part of Clojure where they are used to optimize JavaScript produced by Clojure. Yes, it's a little confusing, even more so when you consider that Clojure is pronounced as "closure".

For those looking for a break from the usual curly braces and C-like syntax, ClojureScript will be a fresh breath of air. It is a dialect of List, and thus has a completely different syntax. Here, take a look at the same example, done in ClojureScript:

```
(ns example)
(defn ^:export square [n]
  (for [x (range n)]
    (* x x)))
(.write js/document
  (reduce str
    (interpose ","
      (square 10))))
```

This too is defining a function called "square" that returns a list of all the squares of all numbers up to "n". The "interpose bit adds a comma between each number, and the reduce bit combines them as strings. The (.write js/document bit is calling the JavaScript "document.write" function.

*GWT, RubyJS and Pyjamas

>>GWT or Google Web Toolkit is a framework for developing client-side web applications using Java. It should be listed above however it is more than just a Java to JavaScript compiler. It provides an entire framework and toolkit, which includes a declarative UI coding language, a UI designer, widgets, etc. that call all be used in Java, but

compile to JavaScript. In the process GWT also optimizes and minimizes your code.

RubyJS and Pyjamas are ports of GWT that work with the Ruby and Python languages respectively. So if you are interested in coding in Java, Python, or Ruby you should check out GWT, Pyjamas and RubyJS respectively.